SENTINEL IPS v2.0

AI-Driven Autonomous Cybersecurity Framework

Project P52 | Team G40 | Global Academy of Technology, Dept. of CSE

A full technical walkthrough for the team — architecture, datasets, models, libraries, results, and the real engineering history behind the project.

Team: Brindha Adiga · Deepa MJ · MT Yashas · Nikita Chavan

Guide: Prof. Sowmya B K | Coordinator: Prof. Vanishree M L | HOD: Dr. R Rajkumar Rao

Document prepared 2026-10-15, covering the project from its first commit to the night before the Anveshanam exhibition.

1. What SENTINEL IPS Actually Is

SENTINEL IPS is an Intrusion Prevention System — software that watches network traffic in real time, decides whether each connection (called a **flow**) is normal or an attack, and then actively responds: it logs the attack, maps it to a known attacker technique, and can block the attacking IP address. It is not just a research model sitting in a notebook — it runs live, captures real packets off a network interface, and has been tested against real attack tools (nmap, hping3, hydra, curl-based exploits) from real attacker machines.

The problem it solves

- Cyber attacks are growing in number and sophistication faster than manual monitoring can keep up with.
- Traditional signature-based tools (match traffic against a list of known bad patterns) miss new or disguised attacks.
- A system is needed that can learn from traffic patterns statistically, not just from a fixed rulebook, and that can react to an attack within seconds, not after a human reviews logs the next day.

The three-part answer

- **Machine learning detection** — an XGBoost model trained on real attack data classifies every network flow as benign or malicious, and (in a second model) names which attack family it belongs to.
- **Signature detection** — fast, exact pattern matching for attacks that have a recognizable payload (SQL injection, XSS, command injection), which ML alone is slower and less certain about.
- **Anomaly + behavioral detection** — catches attacks that don't match any known signature or training example (zero-day-style detection) by flagging statistically unusual behavior.

---

2. Datasets the Model Was Trained On

The model isn't trained on made-up data. It uses two well-known, large, real-world academic intrusion-detection datasets, both captured from real simulated network traffic including real attacks run against a test network.

Dataset	Flows	Features	What's in it
CIC-IDS-2017	2,636,647	82	DoS/DDoS, PortScan, Brute Force, Web Attacks (SQLi/XSS), Botnet, Infi
CIC-IDS-2018	15,007,575	80	Same attack families, captured on a different network/time — used to test

Why two datasets matter: early testing found that a model trained purely on 2017 data scored only ~20% recall when tested on 2018 data — a real, measured **domain shift** problem (attack traffic looks statistically different a year later, on a different network). Training on **both datasets combined** fixed this (recall rose to ~81-97% depending on the test), which is one of the project's genuine research findings, not just an engineering footnote.

---

3. Full System Architecture

The system is organised into layers. Each layer is a real Python package in the repository, not a conceptual diagram only — every module named below exists as actual code.

Core (`core/`)

`preprocessing.py, features.py, flow_collector.py, model.py`

Cleans raw traffic data, selects the most useful features (ANOVA F-test, top 30), assembles raw packets into flows during live capture, and defines the trained model pipeline.

Detection (`detection/`)

`layer1_ml.py, layer2_signatures.py, layer3_anomaly.py`

The three detection layers described in Section 1 — ML classification, signature/payload matching, and IsolationForest-based anomaly scoring.

Intelligence (`intelligence/`)

`threat_feeds.py, ip_reputation.py, domain_reputation.py, honeypot.py`

Checks attacker IPs against a local blacklist and Tor exit-node list (AbuseIPDB/VirusTotal integration exists but needs API keys we don't currently have); runs 5 decoy services (fake SSH/FTP/admin-panel/API) — any connection to these is 100% confirmed malicious by definition.

Attribution (`attribution/`)

`geolocator.py, mitre_mapper.py, threat_profiler.py`

Geolocates the attacker's IP, maps every detected attack to a real MITRE ATT&CK technique ID (the industry-standard attack taxonomy), and classifies the attacker's apparent skill level (script-kiddie vs. organized vs. APT-style) from their behavior pattern.

Response (`response/`)

`ip_blacklister.py, connection_terminator.py, rate_limiter.py, alert_dispatcher.py`

Actually blocks attacking IPs (real Windows firewall / Linux iptables rules), can terminate or rate-limit a connection, invalidate a hijacked session, and dispatch alerts (email/Slack/webhook — wired but no channels configured yet).

Adaptive (`adaptive/`)

`mistake_collector.py, adaptive_trainer.py, zero_day_miner.py`

Collects the model's own mistakes (false positives/negatives) from live traffic and can retrain the model on them, so the system genuinely improves itself over time rather than staying static.

Forensics (`forensics/`)

`packet_logger.py, timeline_builder.py, compliance_reporter.py`

Captures full packet payloads to PCAP files for every live session (so every attack can be replayed and inspected later), and can reconstruct the timeline of an attack sequence.

Explainability (`explainability/`)

`shap_explainer.py, attack_narrator.py, risk_scorer.py`

Uses SHAP (SHapley Additive exPlanations) to show exactly which traffic features drove a decision — not a black box — plus plain-English attack descriptions and a 6-dimension risk score.

Dashboard (`dashboard/`)

`server.py, live_monitor.py, attack_map.py`

A live web dashboard (Flask-SocketIO backend + React frontend) that shows attacks as they happen, with a live world map, MITRE ATT&CK matrix, and system health — see Section 7.

---

4. The Machine Learning Pipeline — Step by Step

This is the heart of the project: how a raw network flow becomes a verdict ("benign" or "attack, 94% confident, type = SQL Injection").

- **1. Chunked loading** — The datasets are tens of millions of rows; loading them all into memory at once would crash a normal laptop. Every dataset read happens in 100,000-row chunks (`pandas.read_csv(..., chunksize=100_000)`).
- **2. Preprocessing** — Missing values are filled with the median (`SimpleImputer`), infinite values (from division-by-zero in rate features) are clipped to a safe range, and 8 known constant/useless columns are dropped.
- **3. Scaling** — Features are standardized (`StandardScaler`) so no single feature dominates just because of its numeric scale (e.g. bytes vs. flag counts).
- **4. Feature selection** — An ANOVA F-test picks the 30 most statistically informative features out of ~80, which both speeds up training and improves the SHAP explanations later (feature names are deliberately preserved through this step, since passing a plain array instead of a labeled DataFrame was an early, now-documented bug that broke explainability).
- **5. Classification** — **XGBoost** (gradient-boosted decision trees) does the actual classification. There are two trained models: a **binary** model (benign vs. attack) and a **multiclass** model (which of 8 attack families it is). Labels are run through a `LabelEncoder` first because the raw class numbers aren't contiguous, which XGBoost requires.
- **6. Confidence thresholding** — a flow only counts as a confirmed attack above a confidence threshold (0.55 normally, lowered to 0.35 when testing across datasets, since cross-dataset confidence runs lower even when the call is correct).

Important: the model is XGBoost only — not Random Forest

Earlier drafts of some project materials mentioned Random Forest as part of the model. This has been checked against the actual code (grepped the whole repository) and corrected — **no RandomForestClassifier exists anywhere in this project.** Only XGBoost is trained and used for classification. If anyone is asked about the model in a review, the accurate answer is "XGBoost, with scikit-learn used for preprocessing/scaling/feature-selection, not for the classifier itself."

---

5. Benchmark Results (the real, measured numbers)

These are not estimates — they come from actual held-out test runs, re-verified independently more than once during the project.

Experiment	Accuracy	Precision	Recall	F1	ROC-AUC
2017 -> 2017 (in-distribution)	99.69%	98.53%	99.68%	99.10%	99.99%
2017 -> 2018 (cross-dataset)	85.64%	74.18%	20.12%	31.66%	92.35%
2017+2018 -> 2018 (combined training)	—	100.00%	80.83%	89.40%	—
Lightweight model (10 features)	99.47%	—	—	98.46%	99.95%

The **cross-dataset row is the single most important finding in the project**: it proves a model can look excellent (99.69%) on its own test set and still fail badly (20% recall) on slightly different real-world traffic — the core motivation for the IEEE paper this project produced, and why combined-dataset training matters.

What SHAP says actually drives a decision

Top 5 features by SHAP importance (out of the real feature set, not guessed):

- **bwd_packet_length_std** (33.82%) — how much the size of backward (server -> client) packets varies.
- **average_packet_size** (17.47%)
- **packet_length_mean** (10.02%)
- **packet_length_std** (6.33%)
- **max_packet_length** (5.61%)

---

6. Attacks Detected — What's Real, Live-Tested, vs. Deferred

Every attack type below has actually been run against the live system from a real attacker VM and confirmed detected on the dashboard — this isn't a theoretical list.

- **Network layer:** DDoS, DoS, Port Scan, Botnet/C2 beaconing
- **Application layer:** SQL Injection, XSS, Command Injection, Path Traversal, Web Shell upload, CSRF
- **Identity/credentials:** Brute Force (FTP/SSH/HTTP), Honeypot hits (any connection to 5 decoy services = 100% confirmed malicious, zero false positives by design)
- **Advanced:** Phishing (malicious URL patterns), Data Exfiltration (large outbound transfer), Session Hijacking (stolen-cookie replay)

Honestly deferred — not built, for good reasons

- **Heartbleed** — the trained model class exists, but there is no vulnerable TLS server in this lab to generate real Heartbleed-shaped traffic against, so it's genuinely untestable here.
- **Ransomware** — the only credible network signal is mass SMB file-share writes, and this lab has no SMB/file-share target. Building a detector with zero ability to test it against real traffic would be a hollow heuristic — deliberately not built rather than faked.

- **Malware C2 via JA3 fingerprinting** — JA3 fingerprints an encrypted TLS handshake, and this lab's target service is plain HTTP; there's no TLS traffic anywhere to fingerprint. Also, JA3 is fundamentally a match-against-known-hashes technique, not a statistical detector like everything else in this project — without a real threat-intel hash database it couldn't be meaningfully tested.
- **Infiltration** — tested specifically and honestly documented as a real limitation: the model currently labels any multi-port probing as PortScan, even slow/internal-looking scans meant to simulate lateral movement. It cannot currently tell external reconnaissance apart from internal post-compromise movement using flow features alone.

---

7. The Live Dashboard & How to Run a Demo

The dashboard is a real-time web page (React frontend talking to a Flask-SocketIO backend running inside the same process as the detection pipeline — no separate database or polling, every new detection is pushed to the browser within about 2 seconds). It shows a live attack map, an attacker detail view with MITRE ATT&CK mapping, and system health metrics.

Three ways to run it

- **Live capture** — captures real packets off a network interface in real time: `python sentinel.py live --interface Wi-Fi` (needs Administrator/root privileges and the Npcap packet-capture driver on Windows).
- **PCAP replay** — replays a previously recorded attack session through the exact same live pipeline and dashboard, with no network capture needed at all: `python sentinel.py live --pcap pcap\<file>.pcap --replay-speed 3 --loop`. This is the method used for the 2026-10-15 exhibition (see Section 8).
- **Health check** — verifies all modules wire together correctly with no real traffic needed: `python sentinel.py health`.

Dashboard URL once running: `http://localhost:5000`

---

8. How We Got Here — The Real Engineering Timeline

This project did not go in a straight line from idea to finished product. It involved real testing against real attacker machines across multiple sessions, and real bugs were found and fixed along the way — this is normal, expected engineering, and worth knowing so nobody is surprised when asked "what problems did you run into?" in a review.

All 13 CLAUDE.md build phases complete

Core pipeline, all detection/intelligence/attribution/response/adaptive/forensics/explainability modules built and wired together; 32/32 modules passing health checks.

Domain-shift discovery (M4 milestone)

First live test against a real attacker VM scored only 1.29% accuracy — the model trained on 2017/2018 CSVs didn't recognize live-captured traffic's flow-timing shape at all. Fixed via adaptive retraining on the real captured traffic; accuracy rose to 99.95%. Several real bugs in the retraining code itself were found and fixed in the process (wrong evaluation columns, non-deterministic column

ordering, a broken sample-weighting feature).

IEEE conference paper written and verified (2026-08-03)

A genuine 6-page IEEE-format paper, base paper Cantone, Marrocco & Bria (IEEE Access 2024), built around the real benchmark-to-live generalization gap as its core finding — not a write-up of existing docs, an actual research contribution with re-verified tables.

Live dashboard rebuilt (2026-08-09)

The original Streamlit dashboard never showed real data (it ran as a separate process with its own disconnected copy of the live monitor). Replaced with the current Flask-SocketIO + React dashboard that shares the live pipeline's actual memory — genuinely real-time.

Multi-host live testing across 3 physical machines (2026-08-20/21 onward)

Coordinated testing where this laptop ran the server/target, and two teammates' laptops ran attacker VMs (Kali) over the real network — not just one machine talking to itself. Found and fixed real bugs: target service only listening on one interface, a DHCP IP collision between two attacker VMs, a quoting bug in a benign-traffic script silently dropping 25% of requests.

Full attack-coverage marathon session (2026-08-22 to 2026-08-25)

Over several sessions: fixed a precedence bug where fast port scans were being mislabeled DoS; built real DDoS multi-source correlation (detecting when 2+ attackers flood the same target together, not just scoring each independently); built Exfiltration and Session Hijacking detection; found and fixed a real SQL-injection-pattern false positive that was mis-flagging ordinary file uploads; confirmed honeypot, Phishing, and Infiltration-vs-PortScan behavior against real traffic. By the end, every CLAUDE.md attack type was implemented and live-tested except the three honestly-deferred ones in Section 6.

Exhibition prep and three-page dashboard design (2026-10-12)

Backlog cleanup, PCAP-replay fallback built (used for the actual 2026-10-15 exhibition), a demo preflight checker script, and a redesigned 3-page dashboard layout — fully spec'd and approved but not yet built in React, deliberately deferred past the exhibition.

Night before the exhibition (2026-10-14/15)

Switched the demo network from a phone hotspot (unreliable at the venue) to a direct Ethernet cable. Found and fixed a genuinely bad cable. Found a deeper, still-unresolved bug: the packet-capture driver (Npcap) cannot see inbound packets specifically on this laptop's Ethernet adapter, even though the operating system receives them fine — tried four real fixes (cable, hardware offload settings, a leftover VirtualBox network filter, Npcap's own repair tool), none worked. Proved it's isolated to that one adapter (Wi-Fi capture works perfectly on the same machine). Decided to present using the PCAP-replay fallback instead of live capture, and to properly fix the Ethernet capture bug after the exhibition, not before.

---

9. Full Technology Stack (what's actually used, verified against the code)

The list below was checked by searching every Python file in the project for real `import` statements — not copied from a wishlist. A library appears here only if the code actually uses it.

Category	Library	What it's actually used for
ML / modeling	xgboost	The classifier itself (binary + multiclass models)
	scikit-learn	Imputation, scaling, ANOVA feature selection, LabelEncoder, pipelines
Data processing	pandas, numpy	All data loading, chunked processing, feature math
Explainability	shap	Feature-importance explanations (SHAP values) behind every decision
Visualization	matplotlib, plotly	Static report plots (confusion matrix, ROC) + live dashboard charts
Network capture	scapy	Live packet sniffing, flow assembly, PCAP read/write
Web / dashboard	flask, flask-socketio	Live dashboard backend + real-time push to the browser
Threat intel	requests	HTTP calls to Tor exit-node list, AbuseIPDB/VirusTotal (needs API keys)
Geolocation	geoip2 (+ fallback)	IP geolocation; falls back to the free ip-api.com REST API since no local GeoLite2 data
Persistence	joblib	Saving/loading trained model files (.pkl)
Legacy / unused	streamlit	Old dashboard, kept in the repo but not part of the live product anymore

Listed in requirements.txt but not actually used anywhere in the code (safe to mention if asked, but don't claim these as part of the working system): pyarrow, seaborn, pyshark, python-dotenv, schedule, rich, tabulate.

---

10. Known Limitations — Say These Honestly if Asked

A reviewer who asks a sharp question and gets an honest, specific answer is far more convinced than one who gets a vague "it works great." These are the real, current limitations:

- The three attack types in Section 6 that were deliberately not built, and why.
- Infiltration (lateral movement) currently gets labeled PortScan — a genuine, measured limit of flow-based detection without payload/behavioral context.
- Windows Firewall enforcement (actually blocking traffic at the OS level, not just logging/blacklisting in memory) has been built and tested but is not turned on by default during demos, to avoid an accidental self-lockout on a shared network.
- Live Ethernet packet capture is currently broken on the demo laptop specifically (a Npcap driver binding issue on one network adapter) — Wi-Fi capture and PCAP replay both work correctly; this is a known open item being fixed after the exhibition, not a flaw in the detection logic itself.
- No live API keys for AbuseIPDB/VirusTotal threat-intel lookups — those integrations are built and wired, just not actively calling out in the current demo configuration.

---

11. Quick Glossary (for anyone newer to networking/ML terms)

Flow — One network conversation between two endpoints (e.g. one HTTP request/response), summarized as a set of statistics (packet sizes, timing, flag counts) rather than raw bytes.

XGBoost — A machine learning algorithm that builds many small decision trees in sequence, each one correcting the previous ones' mistakes — very accurate, fast, and widely used in practice for structured/tabular data like network flow statistics.

SHAP — A method for explaining any ML model's individual predictions by showing how much each input feature pushed the decision toward or away from a given class.

MITRE ATT&CK — An industry-standard catalog of known attacker techniques (e.g. T1110 = Brute Force), used so security teams can describe attacks in a common language instead of ad-hoc names.

Honeypot — A fake, deliberately exposed service (fake SSH server, fake admin panel, etc.) that no legitimate user would ever have a reason to connect to — so any connection to it is treated as 100% confirmed malicious.

Domain shift — When a model trained on one dataset performs worse on a different but related dataset, because the statistical patterns it learned don't fully transfer — the central finding behind this project's IEEE paper.

PCAP — A file format that stores raw captured network packets, so a recorded session can be replayed byte-for-byte later, exactly as it happened.